

## Bits, bytes, hexadecimal y archivos

Un bit no significa nada por sí solos, sino que ocupan una función para interpretar su significado, dicho significado puede variar acorde a que función se mete.

$$\text{ej: } f(01000001) = 65, f(b_7, b_6, \dots, b_0) = \sum_{i=0}^7 b_i 2^i$$

esto quiere decir que si interpretas los bits bajo esta representación, daría 65.

pero puede ser que  $g(01000001) = 'A'$  tranquilamente

dato el rango de  $n$  bits sin  
signo es:  $2^n - 1$   
ej: 8 bits =  $[0, 255]$

Kb, mb, etc. viene del SI y usan potencias de diez:  $1\text{Kb} = 1000\text{bytes}$   
 $1\text{Mb} = 1000\text{Kilobytes}$

pero informalmente usan la potencia a dos  $1024$ , etc. realmente:

$1\text{KiB} = 1024\text{bytes}$  Kibibyte  
 $1\text{MiB} = 1024\text{KiB}$  mebibyte  
 $1\text{GiB} = 1024\text{MiB}$  gibibyte

Base Hexadecimal

dado que  $16 = 2^4$

1 dígito hex = 4 bits

$\Rightarrow 2\text{ hex} = 1\text{ byte}$

$\Rightarrow \left( \begin{array}{ccc} 0101 & 1110 & 0101 \end{array} \right)_2$   
 $\quad \quad \quad 5 \quad \quad E \quad \quad 5$   
 $\quad \quad \quad = 0x5E5$

Conversión  
binario  $\rightarrow$  hex

## ASCII; codificaciones de caracteres y compatibilidad

Sea  $C$  un conjunto de caracteres y  $B$  un conjunto de códigos disponibles, Una codificación sencilla puede ser modelada como:

$$\left. \begin{array}{l} E: C \rightarrow B \\ (E \text{ es una función de } C \text{ hacia } B) \\ \text{ej: } E('A') = 01000001 \end{array} \right\} \text{codificar: significado} \rightarrow \text{representación}$$

esta es la regla a acordar para representar texto.

$$E^{-1} = D : D: B \rightarrow C$$

ASCII (American Standard Code for Information Interchange)

no tenía ñ ni tildes, pues se basaba en el alfabeto gringo.

previamente existían códigos para telégrafos y early computing, pero ASCII apareció como el estándar más temprano de la computación moderna para representar texto e información de control uniformemente.

en ASCII, un carácter ocupa 1 byte, aunque realmente solo define códigos de 7 bits ( $2^7 = 128$ ,  $[0, 127]$ )

esto se debe a que antes los sistemas de telecomunicaciones usaban el bit restante para detectar errores.

ASCII inicialmente no fue pensado para dibujar símbolos visibles, sino para controlar dispositivos o estructurar transmisiones.

En las máquinas de escribir o teletipo, 2 acciones físicas eran diferentes:  
mover el carro al inicio, y avanzar el papel una línea  
de línea

Eso se ve reflejado en CR (carriage return) y LF (line feed)

esto es importante al representar un salto de línea

en ASCII bytes: CR =  $0x0D=13$

LF =  $0x0A=10$

juntos:  $0D0A = CRLF$

CR: Hola  
|  
Hola

LF: Hola  
|

'\n' el runtime lo traduce según el sistema,

LINUX usa solo LF } o igualmente  
Windows usa CRLF

'\n' sería 0A

'\r' " 0D

"\r\n" = 0D 0A

El siguiente paso

Mucha gente empezó a usar el último bit, lo que da [128, 255] disponible.

El problema es que no hubo un extensión universal, por lo que aparecieron varias tablas distintas.

CP437: IBM PC/DOS

similares 150-8859-1 (Latin-1) : pensado para lenguas de Europa Occidental  
Windows-1252 informal

etc

# Los editores de texto

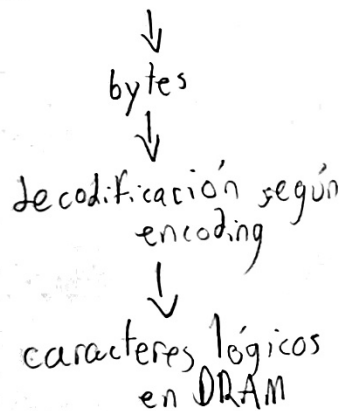
De forma simplificada, un editor de texto hace lo siguiente:  
archivo  $\rightarrow$  bytes  $\rightarrow$  decoder  $\rightarrow$  characters  $\rightarrow$  rendering

Por ahora nos centraremos en bytes  $\rightarrow$  characters

Al abrir un archivo, el editor de texto los trata de convertir en caracteres los bytes y representar su secuencia visual.

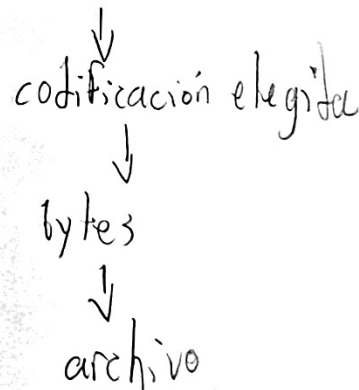
Guardar requiere una decisión de codificación.

abrir =  
? archivo en disco



cuando  
guardas :

caracter X en memoria



o sea puede que  
se decodificó en  
CP437 antes "ñ"  
lo abres en CP437 y  
sigue siendo "ñ"  
pero lo guardas con  
windows-1252 y ahora  
es "±"

reabrir como = reinterpretar  
convertir = codificar como

std::string no almacena texto en X codificación. No impone ASCII, UTF-8 ni nada de eso.

## La llegada de unicode

Usaremos grapheme en vez de character.  
grapheme: unidad básica de un sistema de escritura humano.

(grafema)

El problema previo a unicode era que una compu guarda bits. Entonces ligamos ñ en un era 0xA4 y en otro, 0xF1. El problema es que un mismo carácter podía tener números distintos dependiendo de la tabla.

Unicode lo resuelve haciendo una tabla gigante universal, asignando a cada carácter una identidad numérica.

ej: A  $\rightarrow$  U+0041

ñ  $\rightarrow$  U+00F1

😊  $\rightarrow$  U+1F602

esto es independiente si es Linux, un server, asus, windows, android, etc.

un code point es una de esas posiciones de código  
carácter      code point  
                    ✓ 700F1

es como un ID, vamos.

cosas como esta se pueden representar de 2 formas:

e' → Latin small E with acute  
          233

e' → Latin small E      +      combining acute accent modifier  
          101                                    769

pero OJO, el code point no dice qué byte habrá  
en el archivo: Unicode resuelve "¿Cuál es la identidad de este carácter?"  
no "¿Cómo lo guardo en bytes?"

eso viene en UTF-8  
                  UTF-16  
                  UTF-32

esta es la mejora frente a ascii  
donde el code point = byte

UTF = Unicode transformation Format.

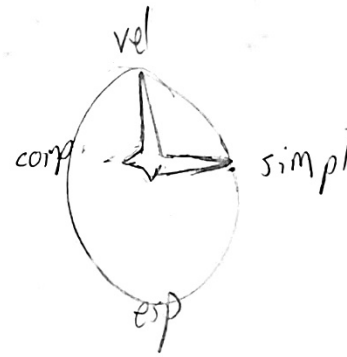
UTF-8  $\rightarrow$  8 bits de code unit.

UTF-16  $\rightarrow$  16 bits " " "

UTF-32  $\rightarrow$  32 bits " " "

UTF-32 usa 32 bits = 4 bytes  
y esencialmente almacena  
todo en 32 bits  
es como un registro constante

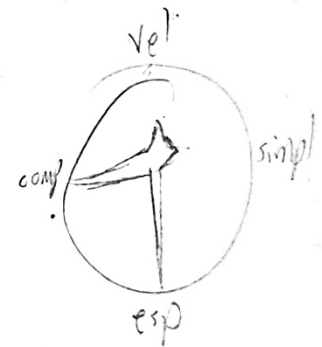
+ velocidad  
+ espacio  
+ simplicidad



UTF-8 un grafema puede  
ocupar de 1-4 bytes  
ej: alfabeto gringo: 1 byte  
No: 2 bytes  
emoj: 4 bytes  
de igual  
el endianness  
en un solo byte  
para saber hasta donde leer

mira el primer  
byte: ...  
0xxxxxx  $\rightarrow$  termina  
aquí (1)  
110xxxxx  $\rightarrow$  ocupa (2)  
1110xxxx  $\rightarrow$  (3)  
11110xxx  $\rightarrow$  (4)

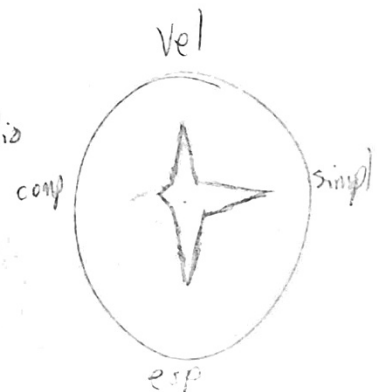
- espacio  
- velocidad  
+ compatible  
ascii



UTF-16 punto intermedio  
16 bits = 2 bytes  
muchas cosas caben ahí  
¿pero y los emojis?

pues usa 2 code  
units ej:  
UTF602 = D83D DE02

- intermedio



¿Qué ocurre al abrir un archivo?

- 1) Obtener los bytes
- 2) Decidir cómo decodificar
- 3) decodificar

Guardar = serializar

gap buffer cerca del cursor para mayor velocidad

## CSV

Comma-separated values (CSV) es un formato de texto para guardar datos tabulares

1024, Monitor, 249.99, 7  
1025, TV, 300.00, 10

} tabular por que  
está en formato  
de tabla

la "," se usa como delimitador/separador de campos.

se suele usar en db y excel.

al igual que archivos binarios, hay que dejar en claro qué representa cada columna.

std::getline

a diferencia de `std::cin`, este no para en espacios; para en `'\n'`. Luego remueve el `'\n'` del stream, o sea, no lo guarda.

`getline(stream, destination);` // para al fallar

este al igual que un cursor binario, continuando la iteración previa terminó.

`std::stoi` y derivadas ignoran espacios y terminan en el carácter final no-parseable.

`getline(file, dest_var)` para ser más específicos.

ó  
`getline(stream_var, dest_var, delimiter)`

por defecto `delimiter = '\n'`

`std::optional<std::vector<std::string>>`

un vector `<string>`  
puede contener

`["Ana", "20", "HN"]`

¿pero qué hago si no puedo  
parsear correctamente dicha  
línea?

`std::optional<T>` lo resuelve sin ambigüedad.

puede estar en: contengo T

no contengo nada

`std::optional<int> x = 5`

`" " <int> y = std::nullopt;`

o sea en nuestro contexto devuelve  
un vector de strings o indica que  
no hay resultado válido.

para un buen CSV parser mínimo  
debe manejar esto:

- campos simples (a,b,c).

- campo vacío (a,c)

- comillas { (a, "b,c", d)

- { ("a", "", "c")